

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

Software Maintenance and Evolution

Literature Review

Code Smell Detection and Analysis

Prepared by

Niaz Rahman
Student ID: 210042159

Antara Arifa Mullick
Student ID: 210042132

Labibah Hoque
Student ID: 210042121

Program: B.Sc. in Software Engineering
Department of CSE

Submitted to: Lutfun Nahar Lota
Submission Date: 22/07/2026

---

1. Introduction

Code smells are structural or design-related problems that may not cause immediate software failure but can make software difficult to understand, modify, test, and maintain. Common examples include Long Method, God Class, Feature Envy, and Data Class. Automating the detection of these problems has been an active area of software engineering research for more than two decades, and the methods used for this purpose have evolved considerably: from fixed metric thresholds and rule-based heuristics, through classical machine-learning classifiers trained on hand-crafted software metrics, to more recent approaches that draw on static analysis tool warnings, neural source-code embeddings, and prompt-based large language models.

This report brings together fifteen studies collected and reviewed by the team across three complementary reading assignments. The first group of five papers examines how dataset quality, class imbalance, smell definitions, source-code representation, and evaluation methodology affect the accuracy and reliability of machine-learning-based code-smell detectors. The second group of five papers traces the evolution of machine-learning-based detection techniques themselves, from one of the earliest classification-based approaches, through feature-engineering strategies built on static analysis tool output and a large comparative evaluation of twenty algorithms, to a prompt-based technique built on large language models and a paper that studies code smells occurring within machine-learning code itself. The third group of five papers turns to applied machine-learning and deep-learning techniques for code-smell detection, covering feature-selection strategies, ensemble stacking, a dedicated Python dataset, deep representation learning with transfer learning, and severity classification under class imbalance.

Read together, these fifteen papers offer a layered picture of the field: focused empirical studies that probe specific factors such as dataset validation, class balancing, and static-analysis features; applied studies that compare feature-selection strategies, ensembling, and severity classification across languages; and papers at the technical frontier that apply prompt learning and neural embeddings, or that turn the lens back onto machine-learning code itself. This report reviews each of the fifteen studies in turn, organised into the three thematic groups described above.

2. Literature Review

The fifteen papers reviewed below are organised into three thematic groups, reflecting the three reading assignments completed by the team. Section 2.1 reviews five papers concerned with dataset quality, feature engineering, and source-code representation in machine-learning-based code-smell detection. Section 2.2 traces the evolution of machine-learning-based detection techniques themselves, from an early classification-based approach to a modern prompt-based method built on large language models. Section 2.3 reviews five papers that apply feature-selection strategies, ensemble learning, deep representation learning, and

severity classification to the practical problem of detecting code smells.

2.1 Dataset Quality, Feature Engineering, and Representation Learning

The following five papers examine how dataset quality, class imbalance, source-code representation, and evaluation methodology affect the reliability of machine-learning-based code-smell detectors.

A Systematic Literature Review on the Code Smells Datasets and Validation Mechanisms

Citation: Zakeri-Nasrabadi, M., Parsa, S., Esmaili, E., and Palomba, F. (2023). "A systematic literature review on the code smells datasets and validation mechanisms." *ACM Computing Surveys*, 55(13s), Article 298, pp. 1–48. DOI: 10.1145/3596908.

Zakeri-Nasrabadi et al. examined 45 code-smell datasets and compared them based on their labelling methods, structure, source projects, availability, and validation quality. The study showed that many datasets were generated using automated detection tools. This is risky because incorrect predictions made by a tool may be included in the dataset as ground-truth labels.

The review found that approximately 96% of the datasets focused on Java, while around 89% were created from open-source systems. Only a small number included industrial software, multiple programming languages, smell-severity information, or rarely studied smells. Many datasets were highly imbalanced, and some did not contain enough clearly identified non-smelly examples.

Another major concern was dataset validation. Approximately 40% of the datasets had no reported validation process, while only part of the datasets were publicly available. Some published dataset links were also no longer accessible.

The paper demonstrates that the reliability of a code-smell detector depends strongly on the quality of the dataset used to train and evaluate it. A model may report high performance while learning from incomplete, inconsistent, or automatically generated labels. The authors therefore recommend larger, diverse, publicly accessible, and manually validated datasets.

A Large Empirical Assessment of the Role of Data Balancing in Machine-Learning-Based Code Smell Detection

Citation: Pecorelli, F., Di Nucci, D., De Roover, C., and De Lucia, A. (2020). "A large empirical assessment of the role of data balancing in machine-learning-based code smell detection." *Journal of Systems and Software*, 169, Article 110693. DOI: 10.1016/j.jss.2020.110693.

Pecorelli et al. investigated whether data-balancing techniques improve machine-learning-

based code-smell detection. Code-smell datasets are commonly imbalanced because non-smelly code elements greatly outnumber smelly elements. As a result, a model may favour the majority class and fail to identify actual smells.

The researchers evaluated five balancing techniques against a no-balancing baseline. These included oversampling, undersampling, the Synthetic Minority Oversampling Technique (SMOTE), cost-sensitive classification, and one-class classification. The experiments covered 11 object-oriented smells and four Model-View-Controller-specific smells, with Naive Bayes used as the base classifier.

The results showed that balancing generally did not produce statistically significant improvements. SMOTE performed better than the other balancing approaches in some cases, but it could not work effectively when the dataset contained very few smelly examples. Oversampling, undersampling, cost-sensitive classification, and one-class classification were mostly ineffective.

The authors also found that class imbalance was not the only cause of poor detection. The structural metrics used by the models often lacked enough information to distinguish smelly code from clean code. Therefore, balancing weak features did not create an effective detector. The study recommends combining structural metrics with textual, semantic, and historical information.

Detecting Code Smells Using Industry-Relevant Data

Citation: Madeyski, L., and Lewowski, T. (2023). "Detecting code smells using industry-relevant data." *Information and Software Technology*, 155, Article 107112. DOI: 10.1016/j.infsof.2022.107112.

Madeyski and Lewowski evaluated machine-learning-based code-smell detection using the MLCQ dataset. This dataset was created from contemporary Java projects and contained more than 14,000 code-smell reviews provided by developers with professional experience.

The study evaluated eight machine-learning techniques and produced 6,720 models. The models were used to detect Long Method, Feature Envy, Blob, and Data Class. Matthews Correlation Coefficient (MCC) was used as the primary evaluation measure because it is more appropriate than ordinary accuracy for imbalanced datasets.

The results varied greatly between smell types. The best median MCC was 0.81 for Long Method, 0.57 for Data Class, 0.51 for Blob, and only 0.31 for Feature Envy. Random Forest achieved the strongest overall performance, while Flexible Discriminant Analysis also performed well.

The study shows that selecting a strong algorithm alone does not guarantee reliable detection. Performance also depends on the smell type, available metrics, severity threshold, dataset composition, and labelling process. Long Method was comparatively easier to detect because it is strongly related to measurable characteristics such as size and complexity. Feature Envy was more difficult because it depends on relationships between methods and other classes.

Although the dataset was labelled by experienced developers, the study remained limited

to Java open-source projects and four smell types. It also did not test the models on a completely independent industrial dataset.

Automatic Detection of Long Method and God Class Code Smells Through Neural Source-Code Embeddings

Citation: Kovačević, A., Slivka, J., Vidaković, D., Grujić, K.-G., Luburić, N., Prokić, S., and Sladić, G. (2022). "Automatic detection of Long Method and God Class code smells through neural source code embeddings." *Expert Systems with Applications*, 204, Article 117607. DOI: 10.1016/j.eswa.2022.117607.

Kovačević et al. investigated whether neural source-code embeddings could improve the detection of Long Method and God Class. Traditional detectors usually depend on manually designed software metrics, which may represent size, coupling, complexity, and cohesion but may fail to capture the semantic meaning of source code.

The study compared traditional software metrics and heuristic detectors with code2vec, code2seq, and CuBERT embeddings. Random Forest, XGBoost, and Support Vector Machine-based Bagging models were trained using the MLCQ dataset. Different class-imbalance techniques were also evaluated.

CuBERT produced the best results, achieving an F1-score of 0.75 for Long Method and 0.53 for God Class. The results suggest that contextual code embeddings can capture semantic information that traditional metrics may overlook.

The improvement was stronger for Long Method than for God Class. God Class remained difficult to detect because it involves broader design responsibilities, cohesion, and architectural context. It is also more subjective than a method-level smell such as Long Method.

This study demonstrates that neural embeddings can improve code-smell detection, particularly when semantic information is important. However, embeddings do not completely solve problems involving subjective labels, class imbalance, dataset quality, and cross-project generalization.

Automatic Detection of Feature Envy and Data Class Code Smells Using Machine Learning

Citation: Škipina, M., Slivka, J., Luburić, N., and Kovačević, A. (2024). "Automatic detection of Feature Envy and Data Class code smells using machine learning." *Expert Systems with Applications*, 243, Article 122855. DOI: 10.1016/j.eswa.2023.122855.

Škipina et al. compared traditional software metrics with CuBERT and CodeT5 embeddings for detecting Feature Envy and Data Class. The study used the MLCQ dataset and evaluated four ensemble-based machine-learning classifiers.

For Data Class, the model based on traditional metrics achieved the best F1-score of 0.62. The strongest CodeT5-based model achieved a nearly equal F1-score of approximately 0.61. This indicates that traditional metrics can represent Data Class effectively because it is

connected to measurable characteristics such as fields, accessor methods, and limited class behaviour.

For Feature Envy, CodeT5 performed better than traditional metrics. However, the overall results remained low. The best CodeT5 model achieved an F1-score of approximately 0.26, while the metric-based model achieved approximately 0.20.

Feature Envy remained difficult because it depends on relationships between a method and the fields or methods of other classes. Existing metric tools did not completely represent these cross-class relationships. The dataset also contained very few positive Feature Envy examples and noticeable disagreement between reviewers.

The study concludes that neural embeddings and traditional metrics have comparable potential, but neither is always superior. Their effectiveness depends on the nature of the code smell being detected.

2.2 The Evolution of Machine-Learning-Based Detection Techniques

The following five papers trace the evolution of machine-learning-based code-smell detection techniques, from one of the field's earliest classification-based approaches to a recent method built on prompt learning and large language models, together with a paper that studies code smells occurring within machine-learning code itself.

Code Smell Detection: Towards a Machine Learning-Based Approach

Citation: Fontana, F. A., Zanoni, M., Marino, A., and Mäntylä, M. V. (2013). "Code smell detection: Towards a machine learning-based approach." In *Proceedings of the 29th IEEE International Conference on Software Maintenance (ICSM 2013)*, pp. 396–399. DOI: 10.1109/ICSM.2013.56.

This paper is one of the earliest attempts to reframe code-smell detection as a supervised classification problem rather than a matter of fixed metric thresholds. The authors were motivated by the observation that different smell-detection tools frequently disagree with one another, which suggests that smells are, at least in part, subjectively interpreted rather than governed by a single objective rule.

The study lays out the practical steps required to build a machine-learning-based detector: selecting a set of object-oriented software metrics as features, assembling a labelled training set from open-source Java systems, and training several classification algorithms to distinguish smelly from non-smelly code elements. The authors compare the resulting models and discuss the trade-offs between the algorithms tested, rather than proposing a single best-performing technique.

Because it sits near the very start of the machine-learning-for-code-smells research line, the paper is candid about the practical difficulties of the task, including the scarcity of large,

reliably labelled datasets at the time and the sensitivity of results to how training examples are selected. Its findings were modest in scope compared with the larger studies that followed, but they established the basic experimental template, extract metrics, label examples, train classifiers, that almost every later paper reviewed in this report continues to follow in some form.

The significance of this paper lies less in its numerical results and more in its role as a reference point: later work, including several of the empirical studies reviewed in Section 2.1, explicitly builds on and extends the methodology introduced here, applying it to larger datasets, additional smell types, and richer feature sets.

On the Adequacy of Static Analysis Warnings with Respect to Code Smell Prediction

Citation: Pecorelli, F., Lujan, S., Lenarduzzi, V., Palomba, F., and De Lucia, A. (2022). "On the adequacy of static analysis warnings with respect to code smell prediction." *Empirical Software Engineering*, 27, Article 69. DOI: 10.1007/s10664-022-10126-5.

Machine-learning-based code-smell detectors are often limited less by the algorithms themselves than by the quality of the features fed into them. This paper investigates whether the warnings produced by existing static analysis tools, already widely used in industry, can serve as useful features for machine-learning-based smell detection, extending an earlier preliminary workshop study by largely the same authors.

The authors design a three-step empirical study. First, they examine whether a measurable relationship exists between static analysis warnings and known code smells, and assess the potential predictive power of these warnings. Second, they build machine-learning models using the most informative combinations of warnings as features. Third, they compare the resulting models against a state-of-the-art code-smell detection approach. The study covers seven types of code smells across twenty-five releases of five open-source projects, giving the findings a reasonably broad empirical base.

The results show that models built from the warnings of individual static analysis tools score between 55% and 91% in terms of F-measure, with the most useful warning types depending on the specific smell being detected. Notably, when the authors replicate their own earlier, smaller-scale study on this larger set of projects, the results do not fully confirm the previous findings: performance, and precision in particular, is considerably lower once a wider and more diverse set of projects is considered.

This drop on replication is itself an important finding: it illustrates how conclusions drawn from a small number of systems can be fragile, and it reinforces a theme that recurs throughout this report, that the external validity of code-smell detection results depends heavily on the diversity of the projects used for evaluation. The paper offers a clear-eyed view of both the promise and the limitations of reusing existing static analysis infrastructure as machine-learning input.

A Study for Method-Level Code Smells Detection Using Machine

Learning Algorithms

Citation: Rao, R. S., Dewangan, S., Mishra, A., and Gupta, M. (2025). "A study for method-level code smells detection using machine learning algorithms." *Systems and Soft Computing*, 7, Article 200346. DOI: 10.1016/j.sasc.2025.200346.

This study zeroes in on four method-level code smells, Long Parameter List, Switch Statement, Feature Envy, and Long Method, which the authors argue have received comparatively less attention than class-level smells such as God Class, particularly with respect to how feature selection and class balancing affect their detection.

The paper runs a comprehensive comparison of twenty different machine-learning algorithms on each of the four smell datasets. An Information Gain feature selection step is used to identify the most relevant metrics, and an Equal Width Discretization approach is applied to address class imbalance before training. Models are evaluated using ten-fold cross-validation across seven performance metrics, including accuracy, precision, recall, F-measure, the Matthews Correlation Coefficient, and the ROC and PRC areas.

The authors report that this is the first study to apply and systematically evaluate feature selection and class balancing specifically for the Long Parameter List and Switch Statement datasets, which have historically received far less methodological attention than Feature Envy and Long Method. Because so many algorithms are tested side by side under a single, consistent methodology, the paper offers a practical reference point for which techniques tend to perform best for each of the four method-level smells.

More broadly, the study provides empirical evidence on how preprocessing choices, feature selection and class balancing in particular, affect final model performance, which is valuable for guiding future automated code-smell detection efforts and for designing experiments that isolate the effect of preprocessing from the effect of the learning algorithm itself.

Prompt Learning for Multi-Label Code Smell Detection: A Promising Approach

Citation: Liu, H., Zhang, Y., Saikrishna, V., Tian, Q., and Zheng, K. (2024). "Prompt learning for multi-label code smell detection: A promising approach." arXiv preprint arXiv: 2402.10398.

This paper introduces PromptSmell, a technique that applies prompt learning, a paradigm built around large pretrained language models, to the task of detecting multiple code smells in the same code element at once. Rather than training a dedicated classifier from scratch on hand-crafted metrics, the authors reframe smell detection as a natural-language task that a pretrained model can be prompted to answer.

Code snippets are first obtained by traversing abstract syntax trees. These snippets are then combined with natural-language prompt templates and mask tokens to construct the input given to the language model, effectively asking the model to fill in the blanks about which smells, if any, are present. To handle the multi-label nature of the task, the authors convert the problem into a multi-classification problem through a label-combination strategy, and

they extend the model's vocabulary with a customised answer space tailored to the code-smell labels being predicted.

A central practical concern the paper addresses is the limited availability of labelled code-smell data. By drawing on the broad, general-purpose knowledge already encoded in a pretrained language model, PromptSmell is shown to perform well even when relatively few labelled examples are available for a given smell, an important property given how small many existing code-smell datasets are. The authors also release a multi-label dataset constructed from real-world Java projects to support further research in this direction.

This paper represents the most recent generation of technique covered in this report, and it illustrates how the broader shift toward large language models and prompt-based methods is beginning to reshape code-smell detection, moving the field away from purely feature-engineering-driven approaches and toward methods that can draw on much larger and more general sources of prior knowledge.

Code Smells for Machine Learning Applications

Citation: Zhang, H., Cruz, L., and van Deursen, A. (2022). "Code smells for machine learning applications." In *Proceedings of the 1st International Conference on AI Engineering: Software Engineering for AI (CAIN '22)*, pp. 217–228. DOI: 10.1145/3522664.3528620.

This paper flips the usual research direction covered elsewhere in this report. Rather than using machine learning to detect code smells, it studies code smells that occur specifically within machine-learning application code, situating itself within the smaller body of "software engineering for machine learning" research rather than the much larger "machine learning for software engineering" body of work reviewed in the rest of this document.

The authors conduct a structured literature search for candidate smells, followed by forward and backward snowballing to widen their pool of sources, and then a full-text review to isolate smells that are genuinely specific to, or especially relevant in, machine-learning code rather than smells that already apply to any general-purpose codebase. Candidate smells are drawn from a mixture of academic papers, grey literature, GitHub commits, and Stack Overflow posts.

The result is a catalogue of twenty-two machine-learning-specific code smells. For each smell, the authors describe the pipeline stage in which it typically arises, for example data preparation, model training, or deployment, the potential problems it can cause if left unaddressed, and a proposed solution or mitigation. Each smell is also linked back to the academic or grey-literature evidence that motivated its inclusion in the catalogue.

This paper offers a valuable contrasting perspective within the report as a whole: it shows that the relationship between code smells and machine learning runs in both directions. Machine learning can be used to detect smells in ordinary code, as most of the other papers reviewed here demonstrate, but machine-learning code itself also has its own distinctive smells that require dedicated attention from data scientists and developers building ML-enabled systems.

2.3 Feature Selection, Ensemble Learning, and Severity Classification

The following five papers apply machine learning and deep learning directly to the practical problem of code-smell detection, ranging from feature-selection strategies and ensemble stacking to a dedicated Python dataset, deep representation learning with transfer learning, and severity classification under class imbalance.

A Novel Approach for Code Smell Detection: An Empirical Study

Citation: Dewangan, S., Rao, R. S., Mishra, A., and Gupta, M. (2021). "A novel approach for code smell detection: An empirical study." *IEEE Access*, 9, pp. 162869–162883. DOI: 10.1109/ACCESS.2021.3133810.

This paper examines four of the most commonly studied code smells, God Class, Data Class, Feature Envy, and Long Method, using datasets built from seventy-four open-source systems. Rather than proposing a single new algorithm, the authors compare six different machine-learning classifiers side by side under a consistent experimental setup.

To identify which code metrics are most useful for spotting each smell, the authors apply two feature-selection methods, Chi-squared testing and a Wrapper-based approach, and then fine-tune each of the six classifiers using grid search to extract additional performance from the chosen features.

The most notable result is that Logistic Regression achieves near-perfect accuracy when detecting Long Method, noticeably higher than the accuracy achieved for the other three smells. This echoes a pattern that recurs throughout this report: smells that can be captured by simple, measurable characteristics such as method length tend to be considerably easier to detect than smells that depend on relationships between classes.

Because the paper tests a modest number of classical classifiers under carefully controlled feature selection and tuning, it functions as a solid empirical baseline for the ensemble and deep-learning approaches discussed later in this section, and its results are referenced directly by the same authors' later work on severity classification, reviewed at the end of this section.

Code Smell Detection Using Feature Selection and Stacking Ensemble: An Empirical Investigation

Citation: Alazba, A., and Aljamaan, H. (2021). "Code smell detection using feature selection and stacking ensemble: An empirical investigation." *Information and Software Technology*, 138, Article 106648. DOI: 10.1016/j.infsof.2021.106648.

This study broadens the scope considerably, training fourteen separate classifiers rather than a small handful, and using Gain Ratio as the feature-selection method to narrow the metrics down to the most informative attributes for each smell.

What sets the paper apart is its use of stacking, an ensemble technique in which the predictions of several base classifiers are fed into a second-layer meta-classifier that learns how to

best combine them. The authors test three different meta-classifiers for this second layer, Logistic Regression, Support Vector Machine, and Decision Tree, rather than relying on a single combination rule.

The results show that stacking consistently improves performance over any individual base classifier, and this improvement holds for both class-level smells such as God Class and method-level smells such as Long Method, indicating that the benefit of ensembling is not confined to one particular type of smell.

This paper is a useful counterpoint to the single-classifier comparisons that dominate much of the literature reviewed elsewhere in this report: it suggests that a meaningful share of the remaining performance gap in code-smell detection can be closed simply by combining existing classifiers more cleverly, without needing new features, new architectures, or additional labelled data.

Python Code Smells Detection Using Conventional Machine Learning Models

Citation: Sandouka, R., and Aljamaan, H. (2023). "Python code smells detection using conventional machine learning models." *PeerJ Computer Science*, 9, Article e1370. DOI: 10.7717/peerj-cs.1370.

Most existing code-smell research, including the two papers reviewed above, relies heavily on Java datasets, leaving a considerable gap for other widely used languages such as Python. This paper addresses that gap directly by constructing an entirely new dataset built from Python code, with one thousand labelled samples for Large Class and a further one thousand for Long Method.

The authors then benchmark six conventional machine-learning models on this new dataset, following a broadly similar experimental design to the Java-based studies discussed elsewhere in this section, which allows for a reasonably direct comparison across languages.

Random Forest turns out to be the strongest performer for detecting Large Class. More importantly, the paper as a whole demonstrates that findings from Java-based research do not necessarily transfer cleanly to other languages, an important consideration for anyone building smell-detection tooling for non-Java codebases.

Within this report, this paper stands out less for algorithmic novelty and more for the dataset it contributes: it is a direct, practical response to the Java-centred bias that recurs across almost every other paper reviewed here, and it shows that closing that gap does not necessarily require new techniques, only new labelled data in the target language.

Code Smell Detection by Deep Direct-Learning and Transfer-Learning

Citation: Sharma, T., Efstathiou, V., Louridas, P., and Spinellis, D. (2021). "Code smell detection by deep direct-learning and transfer-learning." *Journal of Systems and Software*, 176, Article 110936. DOI: 10.1016/j.jss.2021.110936.

Rather than relying on hand-crafted software metrics, as the three papers above do, this study applies deep learning directly to raw source code. The underlying idea is that a deep-learning model may be able to learn useful representations of code structure on its own, without a human first deciding which metrics matter.

The authors also investigate transfer learning, testing whether a model trained to detect one type of smell, or trained on one project, can be reused effectively for a different smell type or a different project, rather than needing to be retrained from scratch each time.

This focus on generalisation is one of the more forward-looking contributions among the papers reviewed in this section, since it speaks directly to how practical these models would be if deployed across many different codebases rather than evaluated only on the project they were trained on.

Because it removes the dependency on predefined software metrics entirely, this paper represents a meaningfully different approach from the feature-selection-driven studies discussed earlier in this section, and it parallels the shift toward neural source-code embeddings and prompt-based methods discussed in Section 2.2, even though it arrives at that shift by a different route.

A Study of Dealing with Class Imbalance Problem with Machine Learning Methods for Code Smell Severity Detection Using PCA Based Feature Selection Technique

Citation: Rao, R. S., Dewangan, S., Mishra, A., and Gupta, M. (2023). "A study of dealing with class imbalance problem with machine learning methods for code smell severity detection using PCA based feature selection technique." *Scientific Reports*, 13(1), Article 16245. DOI: 10.1038/s41598-023-43380-8.

This paper moves beyond the simple question of whether a smell is present and instead tries to classify how severe that smell is, a more fine-grained task that is arguably more useful for developers trying to prioritise their refactoring effort.

A common obstacle in severity classification is that severely smelly examples are much rarer than mildly smelly ones, creating a class-imbalance problem that can bias a model toward the majority class. To address this, the authors combine Principal Component Analysis for feature selection with the Synthetic Minority Oversampling Technique, commonly known as SMOTE, to balance the training data before it is used.

This combined pipeline is then evaluated across multiple machine-learning models, and the results show that pairing PCA-based feature reduction with SMOTE-based oversampling produces more reliable severity predictions than would be possible without addressing the imbalance directly.

Read alongside the same authors' earlier binary-detection study reviewed at the start of this section, this paper shows a clear progression in ambition, from simply flagging whether a smell exists to producing a severity ranking that developers can use to prioritise remediation, and it adds a further data point to the recurring observation elsewhere in this report that

class imbalance is one of the most persistent obstacles in code-smell detection.

3. Overall Comparison and Summary

Table 1 places all fifteen papers reviewed above side by side, organised by the three thematic groups discussed in Sections 2.1 to 2.3, and summarises the focus, method, and headline finding of each.

Table 1: Comparative summary of the fifteen papers reviewed in this report.

Study (Year)	Focus / Smells	Method / Technique	Key Finding
2.1 Dataset Quality, Feature Engineering, and Representation Learning			
Zakeri-Nasrabadi et al. (2023)	Quality of 45 code-smell datasets	Systematic literature review of dataset construction	96% Java-focused; 40% report no validation process
Pecorelli et al. (2020)	Effect of data balancing	Five balancing techniques vs. baseline, Naive Bayes	Balancing rarely helps; weak features are the deeper problem
Madeyski & Lewowski (2023)	Industry-relevant labelling (MLCQ)	Eight ML techniques, 6,720 models, MCC	Performance varies sharply by smell (MCC 0.31–0.81)
Kovačević et al. (2022)	Long Method, God Class	code2vec / code2seq / CuBERT + RF, XGBoost, SVM	CuBERT best (F1 0.75 / 0.53); embeddings help most for Long Method
Škipina et al. (2024)	Feature Envy, Data Class	CuBERT / CodeT5 vs. metrics, four ensemble classifiers	Metrics stay competitive for Data Class; Feature Envy remains hard for all
2.2 The Evolution of Machine-Learning-Based Detection Techniques			
Fontana et al. (2013)	Foundational ML classification	Object-oriented metrics + multiple classifiers	Established the extract-label-train template later work builds on
Pecorelli et al. (2022)	Static analysis warnings as features	Three-step study, seven smells, 25 releases	55–91% F-measure, but results do not replicate at larger scale
Rao et al. (2025)	Method-level smells (LPL, SS, FE, LM)	Twenty algorithms, Information Gain + EWD, 10-fold CV	First systematic feature-selection / balancing study for LPL and SS
Liu et al. (2024)	Multi-label smell detection	AST snippets + prompts + LLM masking (PromptSmell)	Performs well with little labelled data; releases new dataset

Table 1 continued from previous page

Study (Year)	Focus / Smells	Method / Technique	Key Finding
Zhang, Cruz & van Deursen (2022)	Smells within ML code itself	Structured search + snowballing + catalogue	22 ML-specific smells identified across pipeline stages
2.3 Feature Selection, Ensemble Learning, and Severity Classification			
Dewangan et al. (2021)	God Class, Data Class, Feature Envy, Long Method	Six classifiers, Chi-squared + Wrapper FS, grid search	Near-perfect accuracy for Long Method via Logistic Regression
Alazba & Aljamaan (2021)	God Class, Long Method (class- and method-level)	Fourteen classifiers, Gain Ratio FS, stacking ensemble	Stacking consistently outperforms any single base classifier
Sandouka & Aljamaan (2023)	Large Class, Long Method (Python)	New labelled Python dataset, six ML models	First Python-specific dataset; Random Forest best on Large Class
Sharma et al. (2021)	Smell detection without hand-crafted metrics	Deep learning on raw code + transfer learning	Learned representations transfer across smells and projects
Rao et al. (2023)	Severity classification (multiple smells)	PCA feature reduction + SMOTE, multiple ML models	PCA + SMOTE improves severity prediction under imbalance

Even though the fifteen papers span more than a decade and range from tightly focused empirical studies to applied machine-learning and deep-learning techniques, several patterns recur across nearly all of them.

The first is a persistent reliance on a small, Java-centred pool of open-source projects, alongside a growing recognition that this needs to change. Zakeri-Nasrabadi et al. found that 96% of the 45 datasets they surveyed focus on Java, and Dewangan et al. and Alazba and Aljamaan both build their studies on established Java-based smell datasets. This narrow empirical base recurs even in papers that are not primarily about datasets: Madeyski and Lewowski's MLCQ dataset and the MLCQ-based studies by Kovačević et al. and Škipina et al. all draw on Java systems as well. Sandouka and Aljamaan stand out as a direct response to this pattern, constructing an entirely new Python dataset specifically because existing research had so little coverage of non-Java languages, and showing that findings from Java-based studies do not necessarily transfer cleanly to other languages.

The second is that class imbalance is treated as a serious obstacle almost everywhere it is examined, though the results are more mixed than they first appear. Pecorelli et al. (2020)

tested five balancing techniques against a no-balancing baseline for binary smell detection and found no statistically significant improvement, and Rao et al. (2025) reach a broadly similar conclusion through feature selection and discretization. Rao et al. (2023) offer a more encouraging counterpoint from a different task: when the goal shifts from binary detection to severity classification, where imbalance is typically more severe, combining PCA-based feature reduction with SMOTE-based oversampling does produce more reliable predictions. Taken together, these results suggest that simple resampling is not a reliable fix on its own, but that imbalance-handling can pay off when paired with deliberate feature reduction and applied to a task, such as severity classification, where the minority class is especially rare.

The third is that the same handful of smells, God Class, Feature Envy, Long Method, and Data Class, dominate the literature regardless of which slice of it is examined. Dewangan et al. study exactly this set of four, Alazba and Aljamaan cover God Class and Long Method specifically, and even Sandouka and Aljamaan's new Python dataset targets Large Class and Long Method, the closest Python analogues to the same familiar smells. Within this short list, a consistent split emerges: smells that can be captured through size and complexity, such as Long Method, are reliably easier to detect than relationship-based or architectural smells such as Feature Envy and God Class, and this split holds regardless of whether the underlying technique is a classical classifier, a stacking ensemble, a neural embedding, or a prompt-based language model, as the results from Fontana et al., Dewangan et al., Kovačević et al., Škipina et al., and Liu et al. collectively show.

Finally, several papers push the field beyond simple binary detection toward outputs and evaluations that better reflect how the technology would actually be used. Sharma et al.'s transfer-learning experiments test whether a model trained on one smell or project can generalise to another; Sandouka and Aljamaan's new Python dataset tests whether findings generalise across languages at all; and Rao et al. (2023) push toward severity classification, a more actionable output for developers deciding what to refactor first, than a simple yes-or-no label. This concern with generalisation and practical usefulness is largely separate from the accuracy-focused comparisons in Sections 2.1 and 2.2, suggesting that the field is beginning to mature along a second axis, not just how accurately a smell can be detected, but how well that detection holds up outside the exact setting it was trained on.

Read as a whole, the fifteen papers suggest that the field has made steady technical progress, moving from fixed thresholds, to classical machine learning and ensembling, to neural embeddings and deep representation learning, to prompt-based language models, while still working to resolve more fundamental issues of dataset diversity across languages, class imbalance, and the gap between simple binary detection and outputs, such as severity rankings, that developers can act on directly. These recurring gaps, rather than further gains in raw accuracy on the same familiar smells, appear to be where future work in this area would have the most to offer.